



Angular – Challenge Técnico

Web App – Red Social Básica

1. Descripción

Este desafío tiene como objetivo evaluar tus conocimientos en **Angular 18**, **Tailwind CSS 4**, **NgRx** (o Signals Store), patrones de arquitectura y buenas prácticas de desarrollo en aplicaciones modernas.

Además, nos interesa ver tu capacidad para construir una experiencia funcional, modular y mantenible.

2. Objetivo

Crear una aplicación web tipo red social donde un usuario pueda:

- Iniciar sesión.
- Ver un feed de publicaciones.
- Crear posteos y comentarios.
- Persistir el estado localmente.

- Administrar estado con NgRx o Signals Store.

3. Requisitos Principales

3.1 Vistas

Login

El usuario debe poder:

- Loguearse con email y contraseña.
- Validar:
 - Email requerido.
 - Formato de email válido.
 - Contraseña requerida.
- Autenticación simulada (sin backend real).
 - Se permite NgRx con persistencia o `localStorage`.
- Implementar al menos **un método de OAuth simulado** (por ejemplo: botón “Login con Google” que mockea el flujo).
- Redirigir al usuario al Feed al autenticarse.

Feed

Debe permitir:

- Mostrar lista de posteos (mockeados inicialmente).
- Cada post puede incluir:

- Autor.
 - Contenido (texto y/o imagen).
 - Comentarios.
 - Likes (opcional).
- Crear nuevos posts.
 - Crear comentarios sobre posts existentes.
 - Persistencia local (NgRx + localStorage, o solo localStorage).
 - Actualización en tiempo real del timeline.
 - Botón de logout que redirija al login.

3.2 SSR y CSR

- Utilizar **Angular Universal** para habilitar SSR.
- Mantener el CSR al mínimo posible.
- Especificar en el README qué rutas usan SSR.

3.3 Estado Global

Usar **NgRx Store** (o Signals Store) para manejar:

- Estado del usuario autenticado.
- Lista de posts.
- Lista de comentarios.
- Persistencia del estado local.

3.4 Diseño

- Implementar **Tailwind CSS 4**.
- Diseño responsivo.
- Seguir **Atomic Design**:
 - Átomos
 - Moléculas
 - Organismos
 - Templates
 - Pages

3.5 Typescript

- Tipado estricto.
- Todas las interfaces deben estar en una carpeta centralizada (ej: `/interfaces`).

3.6 Arquitectura

- Aplicación modular o basada en Standalone Components.
- Organización recomendada por Atomic Design.
- Rutas organizadas de forma clara (ej: `/app/routes` o `/app/pages`).

4. Requisitos Extra (Opcionales)

Storybook

- Documentar al menos dos componentes:
 - Formulario de Login.
 - Card de Publicación.

Interacciones Extras

Agregar de manera opcional:

- Likes.
- Guardados.
- Compartir.

Deploy

Realizar despliegue en alguno de los siguientes servicios:

- Firebase Hosting.
- Netlify.
- GitHub Pages.
- Vercel (compatible con Angular Universal).

Debe incluirse la URL en el README.

5. Entregables

Repositorio

- Repositorio público en GitHub.
- `README.md` con:
 - Pasos de instalación.
 - Ejecución en modo SSR.
 - Ejecución en modo desarrollo.
 - Prisma no es requerido ya que no hay backend real.

Deploy (si aplica)

- URL accesible públicamente.
- Incluir breve explicación del proceso de deploy.

6. Puntos a Evaluar

Código

- Organización limpia.
- Buenas prácticas Angular.
- Correcto uso de NgRx o Signals.
- Tipado adecuado.
- Componentes pequeños y reutilizables.

UX/UI

- Diseño atractivo.
- Adaptabilidad responsiva.
- Consistencia visual.

Funcionalidad

- Cumple requisitos principales.
- Correcto manejo del estado global.
- Persistencia funcional.
- Validaciones completas.

Extras

- Storybook.
- Deploy exitoso.
- Interacciones adicionales implementadas.

